

CODIFICA DEI CARATTERI:

PRIMO

CONCE ASCII → CODICE DI RAPP. DEI CARATTERI IN CODICE BINARIO, FORMATO DA 7 BIT, A CUI VENIVA AGGIUNTO UN BIT DI PARITÀ.

I PRIMI 32 CODICI → CARATTERI USATI NELLA COMUNICAZIONE TRA IL TERMINALE E IL SISTEMA CENTRALE.

I SUCCESSIVI 96 CODICI → LETTERE DELL'ALFABETO INGLESE (26) MINUSCOLE E MAIUSCOLE, LE CIFRE NUMERICHE DA 0-9, LA PUNTEGGIATURA, PARENTESI E ALTRI CARATTERI.

ESISTONO ESTENSIONI DELL'ASCII BASE CHE VENGONO CHIAMATE **CODE PAGE**, E BISOGNA SPECIFICARNE IL TIPO DI CODE PAGE QUANDO SI CODIFICA UN TESTO, E NON SI POSSONO INSERIRE CARATTERI DI CODE PAGE DIVERSI NELLO STESSO TESTO.

UNICODE → INTRODOTTO PER STANDARDIZZARE LA CODIFICA DEI CARATTERI IN TESTO, SI ASSOCIA ALLO STANDARD UCS (UNIVERSAL CHARACTER SET) ISO/IEC 10646 CHE DEFINISCE POSSIBILI CODIFICHE CONCRETE PER I CODEPOINT.

ASSEGNA CODICI UNIVOCI (CODEPOINT) A CARATTERI, SIMBOLI DI TUTTI I LINGUAGGI SCRITTI E SIMBOLI MATEMATICI.

PRIMA VERSIONE → CODICE A 16 BIT → $2^{16} = 65536$ CARATTERI DISTINTI. (0...FFFF) IN ESADEC.

NON È BASTATO → INTRODOTTI SUCC. 16 NUOVI PLANE SU CODIFICA A 16 BIT PER UN TOTALE DI 1114112 CODEPOINT.

→ **BPM: BASIC MULTILINGUAL PLANE**

I PRIMI 256 CODEPOINT DEL UNICODE BMP → COINCIDONO CON L'ASCII ESTESO LATIN-1 (PRIMI 128 DI ASCII BASE)

ES: A → 0041 (ASCII BASE) ? → 003F (ASCII BASE) È → 00C9 (ASCII ESTESO)

SONO RAPP. TANTISSIME LINGUE ESISTENTI ANCHE MORTE, SIMBOLI VARI (EMOJI).

CODIFICHE DI CODEPOINT:

UCS-2 → USA 2 BYTE PER RAPP. IL VALORE DI CIASCUN UNICODE (LIMITATA ALLA VERSIONE UNICODE 1.0 SU 16 BIT).

UTF-8 → UN'ALTRA CODIFICA PER I CODEPOINT UNICODE A LUNGHEZZA VARIABILE, OUVERO LA RAPP. DEI PRIMI 127 CODEPOINT COINCIDONO CON ASCII BASE, I SUCC. 128 CORRISPONDO ALL'ESTENSIONE ASCII LATIN-1.

→ 8 BIT UCS TRANSFORMATION FORMAT È DI DEFAULT IN MOLTI CONTESTI.

UTF-8 → CODIFICA A LUNGHEZZA VARIABILE: DA 1 A 6 BYTE.

→ TUTTI I BYTE SUCCESSIVO AL PRIMO INIZIANO CON 10.

BIT	BYTE 1	BYTE 2	BYTE 3	BYTE 4	BYTE 5	BYTE 6
7	0xxxxxxx	—	—	—	—	—
11 = 5 + 6	110xxxxx	10xxxxxx	—	—	—	—
16 = 4 + 6 · 2	1110xxxx	10xxxxxx	10xxxxxx	—	—	—
21 = 3 + 6 · 3	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	—	—
26 = 2 + 6 · 4	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	—
31 = 1 + 6 · 5	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

SVANTAGGI: ESSENDO A LUNGHEZZA VARIABILE, È + DIFFICILE SAPERE DOVE INIZIA E TERMINA UN CARATTERE.

VANTAGGI:

- PREFISSI DIVERSI TRA PRIMO BYTE E BYTE SUCC. (NON RISCHIA DI PERDERE L'ALLINEAMENTO).
- NON È LEGATO ALL'ORDINE DEI BYTE.
- COMPATIBILE CON ASCII (SU 7 BIT).
- RISPARM. SPAZIO RISPETTO A QUELLI CON LUNGHEZZA FISSA.

UTF-16: I CODEPOINT DEL BMP, SONO RAPP. COME IN UCS-2 (0000 A FFFF).

→ I CODEPOINT SUGLI ALTRI 16 PLAN SONO SCOMPOSTI IN 2 PAROLE DA 16 BIT:

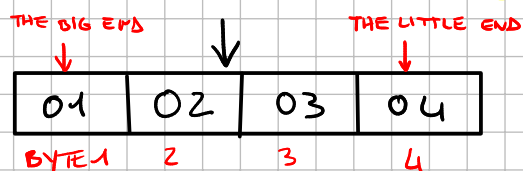
1) D800 + CODEPOINT - 0x10000

2) DC00 + 10 BIT MENO SIGNIFICATIVI DEL CODEPOINT

→ I CODEPOINT: DA D800 A DBFF E DA DC00 A DFFF NON SONO USATI PER DISTINGUERE I CODICI SU 16 BIT DA QUELLI SU 32 BIT.

UTF-32: → UCS-4 → NUMERI SU 32 BIT.

LITTLE ENDIAN VS BIG ENDIAN: → BOM (BYTE ORDER MARK) PREFISSO DA 16 BIT:



• FFFE → LITTLE ENDIAN

• FEFF → BIG ENDIAN

RAPP. IN L.E. → 04 03 02 01 RAPP. IN B.E. → 01 02 03 04

ES: CODIFICARE LA FACCINA CHE RIDE CON GLI OCCHI A CUORE:



→ U+1F60D → ¹0001 | ^F1111 | ⁶0110 | ⁰0000 | ^D1101
17 BIT

UTF-8 → 4 BYTE (16 BIT < 17 BIT < 21 BIT)

1111 0000 | 1001 1111 | 1001 1000 | 1000 1101
FO 9F 98 8D

UTF-16 → CODE POINT - 0x10000 1F60D - 10000 = F60D

CODE POINT - 0x10000 0000 | 1111 | 0110 | 0000 | 1101
20 BIT

PRIMA PAROLA:

1101 1000 0000 0000 +
00 0011 1101 =
1101 1000 0011 1101
D 8 3 D

SECONDA PAROLA:

1101 1100 0000 0000 +
10 0000 1101
1101 1110 0000 1101
D E O D

ES 2: LA SEGUENTE ESPRESSIONE CINESE:

你好

SONO NEL PIANO 0, QUINDI LA CODIFICA È IMMEDIATA, ESSENDO CODICI BMP.
→ "Nǐ Hǎo" → "CIAO" → 0x4F60 & 0x597D

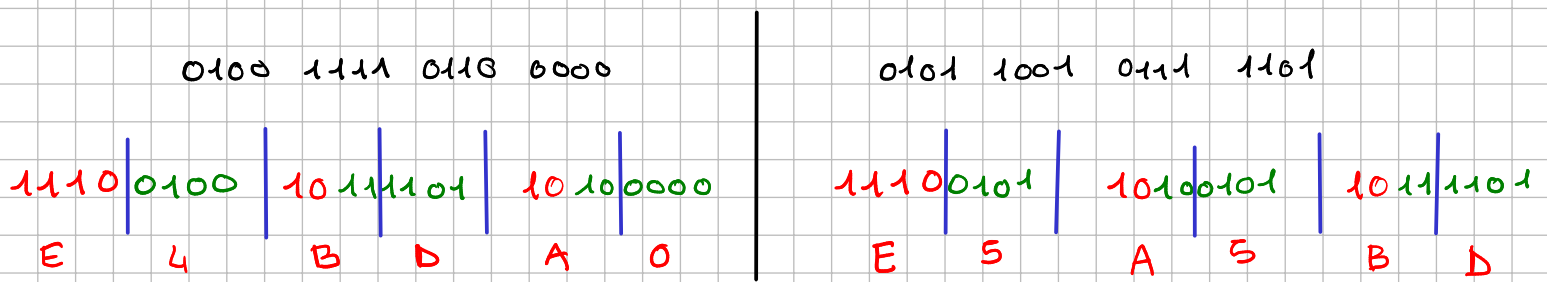
UTF-16:

0100 1111 0110 0000
4 F 6 0

0101 1001 0111 1101
5 9 7 D

BE:	0x4F 0x60	0x59 0x7D
LE:	0x60 0x4F	0x7D 0x59

UTF-8: 16 BIT → RANGE DA 12 BIT A 16 BIT



Non c'è bisogno di ordinamento perché il codice consiste in una sequenza di singoli byte.

0x E4	0x E5
0x B D	0x A 5
0x A 0	0x B D

ES 3:

Domanda 1 – Codifica del testo

Mostrare qual è la rappresentazione in memoria di Ὀδυσσεύς (Odysseus) in greco antico, utilizzando la codifica UCS-2 (Big Endian), basandosi sui codepoint Unicode (del Basic Multilingual Plane) riportati nella tabella seguente. Nota: i codici nella seconda riga sono espressi in esadecimale; ogni cifra esadecimale corrisponde a 4 bit, quindi 2 cifre corrispondono a un byte.

Estratto dalla pagina dei codepoint Unicode (BMP) assegnati ai caratteri del Greco

	Ὀ	δ	β	γ	ε	ζ	η	υ	ύ	λ	μ	ς	σ	τ
Hex	1F48	03B4	03B2	03B3	03B5	03B6	03B7	03C5	03CD	03BB	03BC	03C2	03C3	03C4

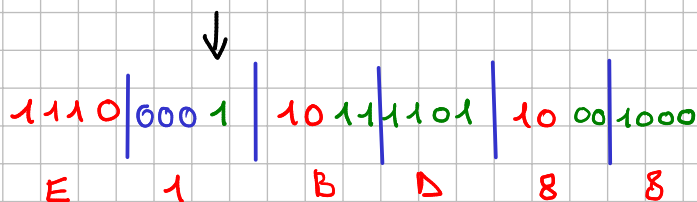
UTF-16 (o UCS-2):

BE:	1F 48	03 B4	03 C3	03 C3	03 B5	03 CD	03 C2
LE:	48 1F	B4 03	C3 03	C3 03	B5 03	CD 03	C2 03

UTF-8 (solo i primi 2):

0x 1F48

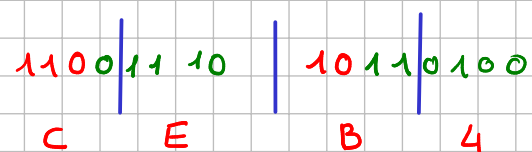
0001 1111 0100 1000
13 BIT → 12-16



0x E1 0x B D 0x 88

0x 03B4

0000 0011 1011 0100
10 BIT → 8-11



0x C E 0x B4

